



# 1 Introduction to Topological Sorting

## 1.1 Definition

Topological sorting is the process of arranging the vertices of a **directed acyclic graph (DAG)** in a linear order such that, for every directed edge  $u \rightarrow v$ , vertex  $u$  appears before vertex  $v$ .

In simple words, if one task must be completed before another task, topological sorting helps us determine a valid order of completing the tasks.

## 1.2 Important Terms

- **Directed graph:** A graph in which every edge has a direction.
- **Directed Acyclic Graph (DAG):** A directed graph that contains no directed cycle.
- **Indegree:** The number of incoming edges to a vertex.
- **Outdegree:** The number of outgoing edges from a vertex.
- **Indegree 0:** A vertex that has no incoming edges.
- **Queue:** A data structure that follows FIFO (First In, First Out).

**Important:** Topological sorting is possible only for a DAG. An undirected graph or a directed graph containing a cycle does not have a valid topological ordering.

## 2 What Is Kahn's Method?

Kahn's method is an algorithm used to perform topological sorting. It follows a BFS-like approach using a queue.

The main idea is:

1. Calculate the indegree of every vertex.
2. Insert all vertices with indegree 0 into a queue.
3. Remove a vertex from the front of the queue.
4. Add that vertex to the topological ordering.
5. Reduce the indegree of each of its adjacent vertices by 1.
6. If any adjacent vertex becomes indegree 0, insert it into the queue.
7. Repeat until the queue becomes empty.

If all vertices are processed, the topological ordering is complete. If some vertices remain unprocessed, the graph contains a cycle.

### 3 Example Graph

Consider the following directed acyclic graph.

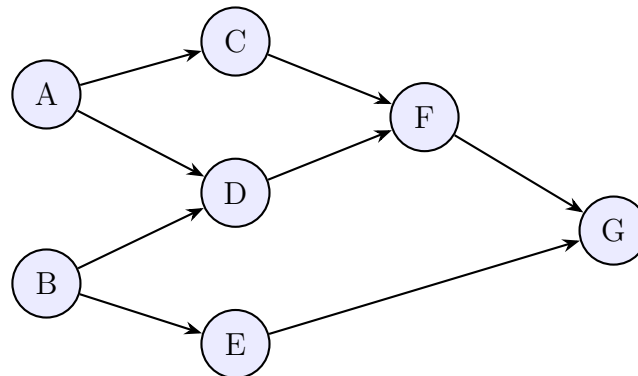


Figure 1: Directed acyclic graph for Kahn's method

The directed edges are:

$$E = \{A \rightarrow C, A \rightarrow D, \\ B \rightarrow D, B \rightarrow E, \\ C \rightarrow F, D \rightarrow F, \\ E \rightarrow G, F \rightarrow G\}$$

We will find a topological ordering using Kahn's method.

### 4 Step 1: Calculate the Indegree

Indegree is the number of edges entering a vertex.

For example:

- A has no incoming edges, so  $\text{indegree}(A) = 0$ .
- D receives edges from A and B, so  $\text{indegree}(D) = 2$ .
- F receives edges from C and D, so  $\text{indegree}(F) = 2$ .
- G receives edges from E and F, so  $\text{indegree}(G) = 2$ .

The initial indegrees are:

| Vertex | Indegree | Incoming Edges                     |
|--------|----------|------------------------------------|
| A      | 0        | None                               |
| B      | 0        | None                               |
| C      | 1        | $A \rightarrow C$                  |
| D      | 2        | $A \rightarrow D, B \rightarrow D$ |
| E      | 1        | $B \rightarrow E$                  |
| F      | 2        | $C \rightarrow F, D \rightarrow F$ |
| G      | 2        | $E \rightarrow G, F \rightarrow G$ |

Table 1: Initial indegree of each vertex

Vertices A and B have indegree 0.  
Therefore, insert them into the queue.

$$\boxed{\text{Queue} = [A, B]}$$

## 5 Step 2: Apply Kahn's Method

We process the queue one vertex at a time.

### Iteration 1: Remove A

- Remove A from the queue.
- Add A to the result.
- A has outgoing edges to C and D.
- Reduce  $\text{indegree}(C)$  from 1 to 0.
- Reduce  $\text{indegree}(D)$  from 2 to 1.
- Insert C into the queue because its indegree is 0.

$$\text{Result} = [A]$$

$$\text{Queue} = [B, C]$$

### Iteration 2: Remove B

- Remove B and add it to the result.
- Reduce  $\text{indegree}(D)$  from 1 to 0.
- Reduce  $\text{indegree}(E)$  from 1 to 0.
- Insert D and E into the queue.

$$\text{Result} = [A, B]$$

$$\text{Queue} = [C, D, E]$$

**Iteration 3: Remove C**

- Remove C and add it to the result.
- C has an outgoing edge to F.
- Reduce indegree(F) from 2 to 1.
- F is not inserted because its indegree is not 0.

$$\text{Result} = [A, B, C]$$
$$\text{Queue} = [D, E]$$
**Iteration 4: Remove D**

- Remove D and add it to the result.
- D has an outgoing edge to F.
- Reduce indegree(F) from 1 to 0.
- Insert F into the queue.

$$\text{Result} = [A, B, C, D]$$
$$\text{Queue} = [E, F]$$
**Iteration 5: Remove E**

- Remove E and add it to the result.
- E has an outgoing edge to G.
- Reduce indegree(G) from 2 to 1.
- G is not inserted because its indegree is not 0.

$$\text{Result} = [A, B, C, D, E]$$
$$\text{Queue} = [F]$$

### Iteration 6: Remove F

- Remove F and add it to the result.
- F has an outgoing edge to G.
- Reduce indegree(G) from 1 to 0.
- Insert G into the queue.

$$\text{Result} = [A, B, C, D, E, F]$$

$$\text{Queue} = [G]$$

### Iteration 7: Remove G

- Remove G and add it to the result.
- G has no outgoing edges.
- The queue becomes empty.

$$\text{Result} = [A, B, C, D, E, F, G]$$

$$\text{Queue} = []$$

## 6 Complete Trace Table

The following table summarizes every iteration.

| Step    | Removed | Indegree Updated | Queue After Step | Result                |
|---------|---------|------------------|------------------|-----------------------|
| Initial | –       | None             | [A, B]           | []                    |
| 1       | A       | C: 1→0, D: 2→1   | [B, C]           | [A]                   |
| 2       | B       | D: 1→0, E: 1→0   | [C, D, E]        | [A, B]                |
| 3       | C       | F: 2→1           | [D, E]           | [A, B, C]             |
| 4       | D       | F: 1→0           | [E, F]           | [A, B, C, D]          |
| 5       | E       | G: 2→1           | [F]              | [A, B, C, D, E]       |
| 6       | F       | G: 1→0           | [G]              | [A, B, C, D, E, F]    |
| 7       | G       | None             | []               | [A, B, C, D, E, F, G] |

Table 2: Kahn's method: step-by-step trace

Therefore, the topological ordering obtained is:

$$A, B, C, D, E, F, G$$

## 7 Algorithm for Kahn's Method

```

KahnTopologicalSort(Graph):
    Calculate indegree of every vertex
    Create an empty queue Q
    Create an empty result list

    for each vertex v:
        if indegree[v] == 0:
            insert v into Q

    while Q is not empty:
        u = remove front of Q
        add u to result

        for each vertex v adjacent to u:
            indegree[v] = indegree[v] - 1

            if indegree[v] == 0:
                insert v into Q

    if number of vertices in result == V:
        print result
    else:
        print "Graph contains a cycle"

```

## 8 Why Is Kahn's Method Similar to BFS?

Kahn's method is called BFS-based because it uses a queue and processes eligible vertices in FIFO order.

However, it is not the same as ordinary BFS.

| Breadth-First Search (BFS)                                   | Kahn's Method                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------|
| Visits vertices level by level.                              | Processes vertices whose indegree is 0.                             |
| A vertex is discovered from a visited vertex.                | A vertex becomes eligible when all its incoming edges are removed.  |
| Used for traversal and shortest paths in un-weighted graphs. | Used to find a topological ordering of a DAG.                       |
| Can be used on directed or undirected graphs.                | Requires a directed acyclic graph for a valid topological ordering. |

Table 3: BFS versus Kahn's method

## 9 Time Complexity

Let:

- $V$  = number of vertices
- $E$  = number of edges

### 9.1 Step 1: Calculate Indegrees

Every vertex is initialized once, and every edge is examined once.

$$T_1(V, E) = O(V + E)$$

### 9.2 Step 2: Process the Queue

Each vertex enters and leaves the queue at most once.

Therefore, queue operations take:

$$O(V)$$

Each directed edge is examined once while processing its source vertex. Updating indegrees for all edges takes:

$$O(E)$$

Thus,

$$T_2(V, E) = O(V + E)$$

### 9.3 Overall Time Complexity

$$\begin{aligned} T(V, E) &= O(V + E) + O(V + E) \\ &= O(2V + 2E) \\ &= \boxed{O(V + E)} \end{aligned}$$

The auxiliary space complexity is:

$$\boxed{O(V)}$$

for the indegree array, queue, and result list, excluding the graph representation.

## 10 What Happens If the Graph Contains a Cycle?

Consider a directed cycle:

$$A \rightarrow B \rightarrow C \rightarrow A$$

Every vertex has indegree 1. Therefore, no vertex has indegree 0.

The queue is initially empty, so no vertex can be processed.

Kahn's method detects a cycle when the number of processed vertices is less than the total number of vertices.

If processed vertices  $< V$ , the graph contains a cycle.

Hence, no valid topological ordering exists for a directed graph containing a cycle.

## 11 Applications of Topological Sorting

- **Course scheduling:** Complete prerequisite courses before advanced courses.
- **Project planning:** Perform dependent tasks in the correct order.
- **Build systems:** Compile required modules before dependent modules.
- **Task scheduling:** Schedule tasks while respecting dependencies.
- **Software installation:** Install dependencies before the software that requires them.

## 12 Quick Revision

| Concept             | Remember                                                  |
|---------------------|-----------------------------------------------------------|
| Topological sorting | Orders vertices while respecting directed edges.          |
| Required graph      | Directed acyclic graph (DAG).                             |
| Indegree            | Number of incoming edges.                                 |
| Starting vertices   | Vertices with indegree 0.                                 |
| Data structure      | Queue (FIFO).                                             |
| Main operation      | Remove a vertex and reduce the indegree of its neighbors. |
| Cycle detection     | Processed vertex count is less than $V$ .                 |
| Time complexity     | $O(V + E)$ with an adjacency-list representation.         |
| Auxiliary space     | $O(V)$ , excluding graph storage.                         |

## Key Takeaway

Kahn's method uses a queue to repeatedly select vertices with indegree 0. After processing a vertex, its outgoing edges are removed by decreasing the indegrees of its neighbors. The process continues until all vertices are ordered or a cycle is detected.